CHANNELSCOPE ANALYTICS

MASTER GOOGLE AI STUDIO DEVELOPMENT PROMPT

YouTube Channel Analytics, Growth Insights & Revenue Estimator

YOUR ROLE

Act as a coordinated team of senior AI engineers, full-stack developers, API integration specialists, UI/UX designers, data visualization engineers, SEO strategists, cybersecurity engineers, technical writers, and QA specialists. Your mission is to **inspect, plan, build, test, audit, and prepare** a complete, production-minded website named **ChannelScope Analytics**.
Do not create merely a landing page, static

dashboard, visual prototype, or conceptual demonstration.
Build a genuinely functional YouTube analytics platform that:

- Searches for publicly accessible YouTube channels without requiring visitors to sign in.
- Retrieves real public channel and video information through YouTube Data API v3.
- Provides transparent, assumption-based revenue estimates.
- Supports optional private owner analytics through authorized Google OAuth and YouTube Analytics API access.
- Displays six interactive Chart.js visualizations.
- Includes all required website pages and ten complete SEO articles.
- Is responsive, accessible, secure, documented, and prepared for Vercel

deployment.

Follow the specifications below. Make sensible engineering decisions without repeatedly asking unnecessary questions.

---

1. PROJECT INSPECTION AND IMPLEMENTATION STRATEGY

Start by inspecting the available Google AI Studio workspace, existing source code, framework, dependencies, and configuration.

Architecture decisions

- Preserve and use an established compatible framework where possible.
- Do not unnecessarily rewrite working code

or remove existing functionality.

- For a new project, prefer Next.js with TypeScript and server-side route handlers, if supported.
- Use React and TypeScript for the interface.
- Use Node.js server-side API integration.
- Use Chart.js for all six required charts.
- Use Lucide or another suitable icon library.
- Use the established styling system or CSS modules/Tailwind CSS.
- Maintain compatibility with Vercel deployment.

If the requested architecture is not supported by the workspace, choose the closest practical alternative and document the difference.

Modular organization

Separate the application into logical modules:

- Application layout and navigation.

- Homepage and search interface.
- Search results and channel selection.
- Channel profile and analytics dashboard.
- Statistics cards and revenue calculator.
- Chart components.
- Video and playlist components.
- OAuth connection interface.
- Blog index and article content.
- Legal and informational pages.
- Reusable API clients and server routes.
- Validation, formatting, and configuration utilities.
- Loading, error, and empty-state components.

Keep API operations, business logic, data formatting, visualization, and presentation separate.

Implementation priority

Build in this order:

1. Working public channel search and channel identification.
2. Real public channel information and dashboard.
3. Video uploads and playlist retrieval.
4. RPM revenue estimator.
5. All six chart components and their genuine data states.
6. Required website pages and SEO articles.
7. Optional authorized owner analytics.
8. Security, accessibility, performance, testing, documentation, and deployment preparation.

Do not abandon the project after generating the interface. Preserve the complete requirements throughout implementation.

---

2. PRODUCT VISION AND DATA INTEGRITY

ChannelScope Analytics is an independent online YouTube channel analytics and revenue-estimation platform.

Primary visitor journey

1. Visitor opens the homepage.
2. Visitor enters a YouTube channel name, URL, handle, or Channel ID.
3. Visitor clicks **See Analytics**.
4. Application identifies the channel.
5. If multiple channels match, visitor selects the correct result.
6. Server retrieves available public information.
7. Dashboard displays the retrieved information.
8. Visitor explores statistics, playlists, videos, charts, and revenue estimates.

9. Channel owners can optionally connect their own YouTube account for authorized private analytics.

Public channel searches must work without requiring visitors to sign in.

Mandatory data classification

Clearly distinguish:
PUBLIC API DATA Information retrieved from YouTube Data API v3.
OWNER ANALYTICS Private analytics retrieved through the YouTube Analytics API after successful OAuth authorization.
ESTIMATED DATA Calculations based on explicitly identified assumptions, supplied RPM values, and available or manually entered view counts.
Never present estimated, unavailable, demo, sample, or manually entered information as

verified live YouTube data.
Every metric must have an appropriate source label or explanation wherever confusion is possible.

---

3. YOUTUBE DATA API V3 INTEGRATION

Implement genuine server-side integration with the official YouTube Data API v3.
Official documentation:

- https://developers.google.com/youtube/v3/docs
- https://developers.google.com/youtube/v3/docs/channels/list
- https://developers.google.com/youtube/v3/docs/search/list

API key security

Store the YouTube API key exclusively on the server.

Environment variable:

YOUTUBE_API_KEY

Create .env.example containing:

YOUTUBE_API_KEY=

Never:

- Hardcode the actual key.
- Expose it in client-side JavaScript or browser bundles.
- Use NEXT_PUBLIC_ or VITE_ prefixes for this secret.
- Commit real credentials to source control.
- Return the key in API responses.
- Print the key in browser logs or error messages.

All YouTube Data API requests requiring this key

must pass through server-side endpoints. Document Google Cloud project creation, API activation, key creation, key restrictions, and quota monitoring.

Supported channel inputs

Accept:

1. Channel ID.
2. YouTube handle, such as @ChannelName.
3. Standard URL: youtube.com/channel/CHANNEL_ID.
4. Handle URL: youtube.com/@ChannelName.
5. Legacy username URL: youtube.com/user/USERNAME.
6. Channel name entered as a search query.
7. Custom /c/ URL where possible through a search fallback.

Validate and normalize input before sending API requests.

Channel lookup strategy

Use the least expensive reliable lookup method first.

Input type	API method
Channel ID	channels.list with id
Handle	channels.list with forHandle
Legacy username	channels.list with forUsername
Name or unresolved custom URL	search.list with type=channel

For search fallback, use:

- part=snippet
- type=channel

- The normalized user query

Do not automatically select the first result when multiple channels match.
Display a channel-selection interface containing:

- Profile thumbnail.
- Channel name.
- Description preview.
- Channel ID.
- Available public information.

Allow the visitor to choose the correct result.
Minimize quota consumption. Cache successful lookups for a sensible period and avoid unnecessary repeated requests for identical queries.

Retrieve channel information

Use channels.list with relevant parts: snippet,statistics,contentDetails,brandingSetting

s,status

Map the returned data into a typed internal channel model.

Retrieve and display available fields:

- Channel ID.
- Channel title.
- Channel description.
- Custom URL or handle.
- Creation date.
- Profile image.
- Channel country, when available.
- Public subscriber count, when visible.
- Lifetime channel views.
- Public video count.
- Hidden subscriber-count status.
- Channel keywords, when available.
- Uploads playlist ID.
- Related public playlist information.
- Relevant public status information.

Do not assume every requested field will be returned.

Missing information and verification

If the API does not return a field, show N/A or a clear unavailable status.
Never invent a value to fill a card.
Do not invent or imply a verified badge. Use:
Not exposed by public Data API
Only report channel visibility or status where supported by retrieved information.
Use evidence-based activity labels:

- Public channel found.
- Recent uploads detected.
- No recent uploads in retrieved sample.
- Activity information unavailable.

Do not declare a channel permanently active or inactive merely because a public channel record

exists or recent uploads are absent.

---

4. VIDEOS, UPLOADS, AND PLAYLISTS

Use the uploads playlist ID returned through:
contentDetails.relatedPlaylists.uploads
Retrieve uploads using:
playlistItems.list
Retrieve relevant video details and statistics using:
videos.list
Display available information:

- Video title.
- Thumbnail.
- Video ID.
- Publication date.

- Views.
- Likes.
- Comments.
- Duration, when retrieved.
- Direct YouTube video link.

Create a clean recent-uploads section.

Date filters

Provide:

- Last 30 days.
- Last 90 days.
- Last 12 months.

Display how many videos were actually examined.
Use sensible page limits and API quota controls. Clearly identify the retrieved sample and its coverage. Never imply that a limited sample represents the channel's complete upload

history.

Public playlists

Retrieve public playlists through the appropriate YouTube Data API endpoint.
Display:

- Playlist title.
- Thumbnail, when available.
- Playlist ID.
- Public video count, when returned.
- Direct playlist link.

Show the uploads playlist separately from other public playlists.
Provide a clear empty state when no additional public playlists are available.

---

5. CHANNEL ANALYTICS DASHBOARD

Build an elegant, responsive analytics dashboard with independent statistic cards.

Required statistic cards

Create all twelve:

1. Channel ID.
2. Channel Name.
3. Subscribers.
4. Lifetime Views.
5. Total Public Videos.
6. Creation Date.
7. Country.
8. Channel Handle.
9. Channel Keywords.
10. Channel Status.
11. Total Revenue Estimate.

12. Monthly Revenue Estimate.

Use readable numbers, sensible abbreviations, tooltips, and access to full values.
Examples:

- 1.25M views, with the exact value available in supporting text or a tooltip.
- 250K subscribers.
- 1.8K public videos.

Use correct numeric formatting. Never confuse unavailable values with zero.

Dashboard controls and metadata

Include:

- Last updated timestamp.
- Data source label.
- Refresh button.
- API availability state.

- Channel page link.
- Shareable channel URL.

Refresh must trigger a genuine fresh server-side API request, subject to sensible caching.

---

6. COUNTRY-BASED RPM AND REVENUE ESTIMATOR

Build a professional, transparent revenue calculator.
Use these exact initial RPM assumptions supplied by the website owner.
Currency: USD.
RPM represents an assumed revenue amount per 1,000 views.

Country	RPM (USD)
United States	7.53
United Kingdom	5.62
New Zealand	5.56
United Arab Emirates	2.33
Pakistan	2.50
India	2.50

Store the values in one centralized configuration file so they can be updated later.
Do not automatically assume that the channel's registered country is its audience market.

Revenue formula

Estimated Revenue = (Views / 1000) × RPM

Calculator controls

Provide:

- Audience market selection.
- RPM display.
- Monthly views input.
- Lifetime views source.
- Calculation period.
- Recalculate action.

Lifetime estimate

Use the channel's public lifetime view count and selected RPM assumption.

Label it:

Illustrative Lifetime Revenue Estimate

Show the selected market, RPM assumption, view count, calculation period, and calculation details.

Monthly estimate

Use actual authorized monthly views when available or a manually entered monthly view figure.
Do not divide lifetime views by an invented number of months.
When a valid monthly view count is unavailable, display N/A until the visitor enters a monthly view figure or connects an authorized channel.
Never present manually entered views as verified API data.

Revenue card design

The lifetime estimate must appear in a prominent futuristic red card:
Background: #ff4d4d
Include:

- Large, bold USD amount.

- Selected country and RPM assumption.
- Calculation period.
- Supporting calculation details.
- Explanatory tooltip.

Display the monthly estimate in a separate card.

Mandatory revenue notice

Display the following notice clearly:
"Revenue figures are illustrative estimates based on selected RPM assumptions and public or supplied view counts. They are not actual channel earnings, verified YouTube revenue, or a guarantee of future income. Actual earnings depend on monetized playbacks, audience geography, content format, advertiser demand, eligibility, and other factors."
Never imply that public views prove monetization is enabled.
Never describe an RPM-based calculation as

actual earnings.

---

7. OPTIONAL YOUTUBE OWNER ANALYTICS

Implement a separate, optional connection flow for channel owners.
Use YouTube Analytics API only after the user explicitly connects and authorizes their own account through Google OAuth.
Official documentation:

- https://developers.google.com/youtube/analytics/channel_reports
- https://developers.google.com/youtube/analytics/reference/reports/query

OAuth scopes

Ordinary analytics:
https://www.googleapis.com/auth/yt-analytics.readonly
Monetary analytics:
https://www.googleapis.com/auth/yt-analytics-monetary.readonly
Request only necessary scopes.
Explain why monetary permission is needed before requesting it.

Authorization rules

The authorizing user must own or have appropriate authorized access to the channel.
OAuth does not grant access to another creator's private analytics.
Monetary information requires appropriate authorization and YouTube Partner Program eligibility.

Security requirements

Implement a secure server-side OAuth flow with:

- OAuth state validation.
- Secure callback handling.
- Secure session management.
- Appropriate access-token and refresh-token protection.
- Encrypted persistent token storage if persistence is implemented.
- Server-side authorization checks.
- Safe token revocation and disconnect behavior.

Never store sensitive OAuth tokens in browser local storage.
Do not create a nonfunctional Connect YouTube button.
If OAuth configuration is incomplete, show a clear setup message. Public channel analytics

must remain operational independently of OAuth configuration.

Available owner reports

Where supported and authorized, retrieve:

- Daily views.
- Subscribers gained.
- Subscribers lost.
- Engagement-related metrics.
- Geographic audience breakdown.
- Monthly revenue.
- Other supported channel reports.

Use only report dimensions and metrics actually supported by the API.
Handle unavailable reports, missing permissions, API failures, and channels outside the YouTube Partner Program appropriately.
Never expose one user's private analytics to another user.

---

8. EXACTLY SIX INTERACTIVE CHART.JS VISUALIZATIONS

Implement exactly six chart sections using Chart.js.
Every chart must use real, appropriate data when available. Use responsive sizing, readable labels, useful tooltips, legends where appropriate, and consistent formatting.
Use clean white or light-neutral chart surfaces with sufficient contrast against the dark dashboard.

Chart 1 — Subscriber Growth

Type: Line Chart
Display authorized historical owner analytics:

- Subscribers gained.
- Subscribers lost.
- Net subscriber change.

Do not reconstruct historical total subscribers from an invented baseline.
For public-only channels, display a clear locked or unavailable state.

Chart 2 — Video Uploads

Type: Bar Chart
Use actual publication dates from retrieved uploads.
Group videos by month and display:

- Monthly upload count.
- Selected time range.
- Number of sampled videos.

Clearly identify sample coverage when only recent uploads have been retrieved.

Chart 3 — Views Over Time

Type: Pie Chart

When authorized owner reports provide historical views, show the distribution across meaningful time buckets.

Explain that each slice represents that period's share of views.

Do not imply that a historical breakdown exists when no historical data is available.

Chart 4 — Revenue Trends

Type: Doughnut Chart

When authorized owner revenue reports exist, show monthly revenue distribution.

When only the RPM calculator is available, never describe estimated values as actual revenue.

If manually entered monthly views are used, label the visualization:

Illustrative RPM Scenario

Chart 5 — Engagement Metrics

Type: Radar Chart

Use genuine available information from sampled public videos or authorized owner reports.

Possible metrics:

- Likes per 1,000 views.
- Comments per 1,000 views.
- Video view performance.
- Upload consistency.

For public samples, explain sample size and calculation methodology.

Normalize metrics appropriately before comparison.

Never fabricate likes, shares, comments, or engagement rates.

Chart 6 — Audience Demographics

Type: Polar Area Chart

When authorized YouTube Analytics reports provide audience views by country, display the retrieved data.
For public-only channels, clearly state that actual audience geography is private.
Never mistake a channel's registered country for its audience's geographic location.

Demo Mode

A clearly identified Demo Mode may preview all six chart designs.
It must remain visibly separate from live channel results and always display:
DEMO DATA — NOT REAL CHANNEL ANALYTICS
Never silently substitute demo data for unavailable real statistics.

Chart implementation requirements

- Re-render when source data changes.

- Destroy previous Chart.js instances before replacing them.
- Prevent duplicate canvas rendering.
- Handle chart resizing correctly.
- Avoid layout shifts.
- Handle empty datasets cleanly.
- Provide accessible text summaries.
- Provide a data table or textual alternative for important chart information.

---

9. UI/UX AND VISUAL BRAND

Create a premium futuristic data analytics interface.
The experience must be modern, coherent, professional, and easy to navigate. Avoid generic AI dashboard styling.

Brand

Name:

ChannelScope Analytics

Positioning:

YouTube Channel Analytics & Revenue Estimator

Maintain an independent identity. Do not imply official affiliation with YouTube or Google.

Color palette

- Main background: #0f0f23
- Primary text: #ffffff
- Neon cyan: #00ffcc
- Revenue red: #ff4d4d
- Orange accent: #ff7e5f
- Banner gradient: #ff7e5f to #feb47b
- Secondary surfaces: suitable dark blue-gray shades.
- Chart surfaces: white or light neutral.

Typography

- Headings: Orbitron, bold.
- Body: Poppins, regular or medium.

Load fonts efficiently and provide appropriate fallbacks.

Header

Create a minimal fixed or sticky header where appropriate.
Left:

- ChannelScope Analytics logo.
- Small product descriptor.

Right:

- Home.
- Analytics.
- Blog.

- About.
- Contact.

Do not invent social-media profile URLs. Add social links only when genuine destination URLs are configured.

Hero section

Use a prominent warm orange-to-red gradient banner.
Heading:
YouTube Analytics
Subtitle:
Get detailed insights into any YouTube channel.
Include a central search interface with:

- Input field.
- Search icon.
- Placeholder: Enter channel name, URL, @handle or Channel ID
- Green See Analytics button.

- Helpful search examples.

Search behavior

- Enter key submits.
- Button triggers the same action.
- Disable duplicate requests while loading.
- Show clear loading feedback.
- Show channel selection when multiple matches exist.
- Preserve input after recoverable errors.
- Display understandable failure and empty states.

Dashboard layout

Use a responsive 12-column layout.
Group related information logically. Make key information immediately understandable through clear headings, strong number hierarchy, restrained accent colors, and consistent spacing.

The revenue card must stand out without overwhelming the dashboard.

Charts

Use a responsive chart-card grid that remains readable on mobile.

Footer

Include:

- About.
- Contact.
- Privacy Policy.
- Terms of Service.
- Disclaimer.
- Blog.

Use a contrasting footer surface consistent with the dark visual identity.
Display this independence notice:

"ChannelScope Analytics is an independent analytics and estimation tool and is not affiliated with, endorsed by, or officially connected to YouTube or Google."

Motion and interactions

Use restrained transitions and hover effects.
Do not sacrifice performance for animation.
Respect reduced-motion preferences.

---

10. REQUIRED WEBSITE PAGES

Implement every page with its own proper URL and unique metadata.
Required routes:

1. Home / YouTube Analytics Tool.
2. About Us.

3. Contact Us.
4. Privacy Policy.
5. Terms of Service.
6. Disclaimer.
7. Blog Index.
8. Individual Blog Articles.

Integrate the analytics functionality into the homepage.
Do not create empty placeholder pages.

About Us

Explain:

- What ChannelScope Analytics does.
- How public channel data is retrieved.
- How RPM estimates are calculated.
- The difference between public statistics and private owner analytics.
- The tool's limitations.

Contact Us

Create a professional contact experience.
Do not invent a business address, phone number, or contact email.
Use a configurable contact email or correctly configured form service.
If no mail service is configured, do not claim that a submission was successfully delivered.

Privacy Policy

Accurately describe:

- Data collected.
- Search request processing.
- API usage.
- OAuth permissions, if enabled.
- Token handling.
- Cookies and local storage, if used.
- Analytics integrations, if enabled.

- Third-party services.
- Data retention and deletion.
- Contact process.

Do not claim that no information is collected when the application processes or stores data.

Terms of Service

Cover:

- Appropriate use.
- API dependencies.
- Service availability.
- Data accuracy and limitations.
- User responsibilities.
- Third-party services.
- Liability and other relevant usage limitations.

Disclaimer

Explain that:

- Public data may be limited or rounded.
- Revenue figures are estimates.
- Public statistics do not reveal actual earnings.
- Private analytics require owner authorization.
- The application is independent and not officially affiliated with YouTube or Google.

Write accurate, readable content. Do not claim guaranteed legal compliance.

---

11. TEN COMPLETE SEO BLOG ARTICLES

Create a genuine SEO-focused blog containing ten unique, useful, original articles.
Save each article as an individual content file

and provide its own accessible URL.
Do not generate thin articles filled with repeated keywords.

Required article topics

Article 1: How to Check Any YouTube Channel's Statistics

Cover channel IDs, handles, public information, channel searches, and public-data limitations.

Article 2: YouTube Channel Analytics Explained: A Complete Guide

Cover views, subscribers, videos, upload trends, and available metrics.

Article 3: YouTube Subscribers, Views, and Video Count Explained

Explain what public channel statistics represent and their limitations.

Article 4: YouTube RPM vs CPM: What's the Difference?

Explain RPM, CPM, monetized playbacks, and

their different uses.

Article 5: How to Estimate YouTube Channel Earnings

Explain the RPM formula, market assumptions, calculation examples, and estimate limitations.

Article 6: Public vs Private YouTube Analytics: What Can You See?

Explain YouTube Data API, YouTube Analytics API, and owner authorization.

Article 7: How to Track YouTube Channel Growth Over Time

Explain historical data, snapshots, daily subscriber changes, and data collection.

Article 8: YouTube Shorts vs Long-Form Video Analytics

Explain relevant measurement differences without inventing earnings comparisons.

Article 9: How Audience Location Can Affect YouTube RPM

Explain why audience geography differs from a

channel's registered country and why RPM varies.

Article 10: How to Calculate YouTube Engagement Rate Using Public Data

Explain views, likes, comments, sampling, engagement formulas, and limitations.

Requirements for every article

Each article must contain:

- Unique SEO title.
- Unique meta description.
- One clear H1.
- Logical H2 and H3 headings.
- Original introduction.
- Useful, detailed, accurate content.
- Practical examples where relevant.
- Internal links to the analytics tool and related articles.
- Useful conclusion.

- Relevant FAQs only where they add value.
- Appropriate structured data.
- Canonical URL.
- Social metadata.

Target approximately 800–1,200 useful words per article when the subject supports that length. Do not pad content merely to reach a word count.
Avoid keyword stuffing, repetitive introductions, fabricated statistics, fake author credentials, invented references, and unsupported claims.
Use the editorial identity:
ChannelScope Editorial Team
Do not invent a real person's credentials.
Publication and update dates must reflect actual content-management events.

---

12. TECHNICAL SEO

Implement:

- Unique page titles.
- Unique meta descriptions.
- Canonical URLs.
- Open Graph metadata.
- Twitter/X card metadata.
- Correct heading hierarchy.
- Semantic HTML.
- Image alt text.
- Descriptive URLs.
- Internal links.
- XML sitemap.
- robots.txt.
- Favicon.
- Relevant structured data.
- Custom 404 page.
- Proper redirects.

- Mobile-friendly layouts.
- Crawlable public content.
- Search-friendly article URLs.

Sitemap

Create a valid, dynamically maintained sitemap containing all intended indexable public pages and blog articles.
Exclude:

- Private callback routes.
- Private APIs.
- Query-parameter variations.
- Non-indexable pages.

robots.txt

Create a valid robots.txt file.
Do not accidentally block important public pages or required assets.
Do not expose private endpoints or sensitive

information.

Canonical domain

Use the actual configured production domain.
Never invent a final domain.
If the production domain is not configured, use a clearly documented configuration variable and explain how to set it during deployment.

Structured data

Use only relevant, valid schema types, such as:

- WebSite.
- Organization.
- Article or BlogPosting.
- BreadcrumbList.

Do not add artificial ratings, fake reviews, unsupported claims, or inaccurate schema properties.

Google Search Console readiness

Prepare:

- Sitemap.
- robots.txt.
- Canonical tags.
- Verification-token configuration.
- Indexability checks.

Do not invent a Search Console verification token.
Document how to add the genuine verification token and submit the sitemap.

---

13. RESPONSIVE DESIGN AND ACCESSIBILITY

Ensure the website works properly on:

- Small mobile phones.
- Large mobile phones.
- Tablets.
- Laptops.
- Desktop monitors.

Check for:

- Horizontal overflow.
- Overlapping elements.
- Broken navigation.
- Text clipping.
- Unreadable dashboard cards.
- Chart overflow.
- Incorrect mobile spacing.
- Buttons too small for touch.
- Difficult-to-use form controls.

Implement:

- Semantic HTML.
- Appropriate ARIA labels.
- Keyboard-accessible controls.
- Visible focus indicators.
- Correct color contrast.
- Associated form labels.
- Accessible error messages.
- Image alternative text.
- Reduced-motion support.

Do not sacrifice accessibility for futuristic styling.

---

14. PERFORMANCE AND SECURITY

Implement appropriate safeguards:

- Efficient server-side API handling.
- Sensible response caching.
- Request validation.
- Error normalization.
- Rate limiting where appropriate.
- Secure secret management.
- Safe URL parsing.
- Input sanitization.
- Request timeouts.
- Graceful network failure handling.
- Lightweight frontend bundles.
- Efficient font loading.
- Responsive image handling.
- Chart cleanup and memory management.

Protect the application from accidental YouTube API quota exhaustion.

Never expose:

- API keys.

- OAuth tokens.
- Database credentials.
- Other private configuration.

Never store sensitive OAuth tokens in browser local storage.
Do not add unnecessary accounts, tracking, advertising, payment systems, or databases.
Only implement external integrations when genuinely required by the specified functionality.

---

15. API ERRORS AND USER FEEDBACK

Provide clear user-facing handling for:

- Invalid API key.
- API disabled.

- Quota exceeded.
- Channel not found.
- Ambiguous search results.
- Private or unavailable channel.
- Hidden subscriber count.
- Missing statistics.
- Network timeout.
- Rate limiting.
- Invalid URL.
- No search results.
- OAuth authorization failure.
- Missing OAuth permissions.
- Unavailable historical analytics.
- YouTube Analytics API access denied.
- Unexpected API responses.

Use understandable messages. Never expose raw stack traces, credentials, or sensitive information.
Provide retry options where appropriate.

Never display a successful analytics dashboard when the required data request has failed.

---

16. DOCUMENTATION AND CONFIGURATION

Generate all required application source code and documentation.
Do not stop after providing a plan, a landing page, or a few example snippets.

.env.example

Include required variable names without actual secret values:

- YOUTUBE_API_KEY
- GOOGLE_CLIENT_ID
- GOOGLE_CLIENT_SECRET

- GOOGLE_OAUTH_REDIRECT_URI
- TOKEN_ENCRYPTION_KEY
- DATABASE_URL only if a database is implemented.
- NEXT_PUBLIC_SITE_URL or the equivalent public site URL setting.

Document which variables are mandatory for public analytics and which are optional for owner analytics.

README.md

Include:

- Project overview.
- Technology stack.
- Installation instructions.
- Development commands.
- Production build commands.
- API configuration.
- Google Cloud setup.

- OAuth setup.
- Vercel deployment.
- Environment variables.
- Troubleshooting.
- Security considerations.

API_SETUP.md

Document:

- Enabling YouTube Data API v3.
- Creating and restricting the API key.
- Secure configuration.
- Separate OAuth setup for owner analytics, if implemented.

SEO_SETUP.md

Document:

- Sitemap location.
- robots.txt location.

- Canonical configuration.
- Page metadata.
- Search Console verification.
- Sitemap submission.
- Indexability checks.

IMPLEMENTATION_AUDIT.md

Create a genuine completion audit using exactly these status categories:

- PASS.
- FAIL.
- NOT VERIFIED.

Include evidence for every claimed test or implementation result.
Never fabricate test results.

---

17. TESTING AND ACCEPTANCE CRITERIA

Do not declare the project complete until the application has been tested as far as the available environment permits.

Functional tests

Verify:

1. Channel ID search.
2. @handle search.
3. Channel URL search.
4. Channel name search.
5. Multiple search matches.
6. Channel selection.
7. Channel information display.
8. Missing-data states.
9. Public playlist retrieval.
10. Recent uploads retrieval.

11. Revenue calculations.
12. Monthly views input.
13. RPM market selection.
14. All six chart components.
15. Demo Mode separation.
16. Refresh functionality.
17. Error recovery.
18. OAuth flow, when fully configured.
19. All required page routes.
20. Contact functionality, when configured.

Data integrity tests

Confirm:

- Real API values are mapped correctly.
- Missing values remain unavailable.
- Hidden subscriber counts are handled correctly.
- Revenue formulas calculate correctly.
- Monthly estimates never use invented

monthly views.

- Demo values never appear as live data.
- Historical metrics appear only when available.
- Audience geography is not inferred from channel country.
- Private analytics are restricted to their authorized owner.

UI and accessibility tests

Inspect mobile, tablet, and desktop layouts. Test navigation, search, buttons, cards, dropdowns, charts, loading states, error states, and empty states.

Security tests

Verify:

- API keys are absent from frontend bundles.
- No secrets are committed.

- OAuth tokens are handled server-side.
- Private analytics are protected.
- API errors do not expose credentials.
- Input validation is implemented.
- Public endpoints have appropriate protections against uncontrolled repeated requests.

SEO tests

Verify:

- Required pages load.
- Page titles are unique.
- Meta descriptions are present.
- Canonical URLs are correct.
- Sitemap is accessible.
- robots.txt is accessible.
- Structured data is valid.
- Internal links work.
- Intended public content is crawlable.

Never mark a test as PASS unless it was actually executed.
Clearly identify anything that remains unverified.

---

18. DEPLOYMENT AND PRODUCTION HANDOVER

Prepare the project for Vercel deployment.
Verify, where the environment permits:

- Production build.
- Correct build and start configuration.
- Environment-variable documentation.
- Server-side API routes.
- Secure configuration.
- Production URL configuration.
- Sitemap access.

- robots.txt access.
- Canonical URLs.
- Error pages.
- Mobile rendering.
- Real API functionality.

Do not claim a deployment succeeded unless it was actually completed and verified.
Provide concise instructions for:

1. Creating a Google Cloud project.
2. Enabling YouTube Data API v3.
3. Creating and restricting an API key.
4. Adding environment variables.
5. Deploying to Vercel.
6. Testing real channel searches.
7. Configuring optional OAuth.
8. Submitting the sitemap to Google Search Console.

19. NON-NEGOTIABLE IMPLEMENTATION RULES

- Build the actual functional website.
- Do not return only a design concept.
- Do not omit required pages.
- Do not omit any of the six chart components.
- Do not replace real API integration with hardcoded sample data.
- Do not display fake subscribers, views, revenue, or demographics.
- Do not create nonfunctional controls.
- Do not invent API properties.
- Do not expose API secrets.
- Do not claim access to private analytics without OAuth authorization.
- Do not silently substitute demo data for live

data.

- Do not create thin, repetitive SEO articles.
- Do not claim tests passed without evidence.
- Do not make unnecessary changes outside the project scope.
- Preserve existing functionality when modifying an established project.
- Prefer targeted fixes over unnecessary rewrites.
- Document any incomplete feature and its exact blocker.

If the environment lacks a required credential, service, or capability, implement everything that can genuinely be completed, clearly identify the blocker, and document the remaining setup steps.
Do not simulate a successful integration or claim functionality that has not been implemented.

---

20. FINAL DELIVERABLES

At completion, provide the following:

1. Complete working website source code.
2. Functional public channel search.
3. Secure server-side YouTube Data API integration.
4. Public channel information dashboard.
5. Playlist and recent-video sections.
6. Country-based RPM calculator.
7. Monthly and lifetime estimate displays.
8. Exactly six interactive Chart.js visualizations.
9. Optional authorized owner analytics implementation or a clearly documented setup state.
10. Homepage and all required informational

pages.

11. Ten complete, original SEO articles.
12. SEO metadata and relevant structured data.
13. Working sitemap.xml and robots.txt.
14. Responsive desktop, tablet, and mobile interface.
15. Secure API and environment configuration.
16. README.md.
17. API_SETUP.md.
18. SEO_SETUP.md.
19. IMPLEMENTATION_AUDIT.md.
20. Vercel deployment instructions.

FINAL EXECUTION INSTRUCTION

Start by inspecting the actual project workspace and selecting the best compatible architecture.

Then implement the application directly in the workspace.
Do not stop at planning, wireframes, screenshots, sample code, or a static interface.
Build the complete public analytics experience first. Then implement optional owner-only analytics wherever the necessary configuration and capabilities are available.
After implementation:

1. Inspect the actual source code.
2. Run available tests and the production build.
3. Check the actual functionality.
4. Fix defects that can be resolved in the environment.
5. Audit security, data integrity, responsiveness, and SEO.
6. Prepare the documentation and production handover.
7. Record PASS, FAIL, and NOT VERIFIED results

honestly.

The finished application must be an original, professional, secure, responsive, SEO-ready, genuinely functional YouTube analytics website.
QUALITY STANDARD: Production-minded engineering, transparent data, useful original content, premium interface, and verifiable functionality.